

Universidad de Oriente	
Machine Learning	Clase 10 - Ejemplo 1 - Regresión de Ridge, Lasso y Red Elástica

```
In [5]: import pandas as pd
import os
import tarfile
import urllib.request
import numpy as np
from pandas.core.common import flatten
from plotnine import *
from array import *
import scipy.stats as stats
import math
import matplotlib as mpl
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn import linear_model
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms
```

Ejemplo 1 - Parte 1

En este ejemplo se profundizará en los modelos de regresión penalizada. Cargar el dataset de ventas y publicidad, y realizar un modelo de regresión Ridge del volúmenes de las ventas con las variables restantes.

- Con el parámetro de encogimiento $\alpha=10$
- Con el parámetro de encogimiento $\alpha=100$
- Con el parámetro de encogimiento $\alpha=1000$
- Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?
- Realizar la validación cruzada para determinar el valor de α que proporciona un menor error cuadrático
- Comparar el comportamiento de los coeficientes cuando α disminuye y cuando α tiende a hacerse más grande.

```
In [7]: #Cargar La base de datos

datos = pd.read_csv("https://raw.githubusercontent.com/rpizarrog/Analisis-Inteli
```

```
In [11]: #Mostrar Los datos
datos
```

Out[11]:

	Unnamed: 0	X	TV	Radio	Newspaper	Web	Sales
0	1	1	230.1	37.8	69.2	306.634752	22.1
1	2	2	44.5	39.3	45.1	302.653070	10.4
2	3	3	17.2	45.9	69.3	49.498908	9.3
3	4	4	151.5	41.3	58.5	257.816893	18.5
4	5	5	180.8	10.8	58.4	195.660076	12.9
...	...	...	...	...	...	...	...
195	196	196	38.2	3.7	13.8	248.841073	7.6
196	197	197	94.2	4.9	8.1	118.041856	9.7
197	198	198	177.0	9.3	6.4	213.274671	12.8
198	199	199	283.6	42.0	66.2	237.498063	25.5
199	200	200	232.1	8.6	8.7	151.990733	13.4

200 rows × 7 columns

In [13]: *#Información de las variables*
datos.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0  200 non-null   int64
1   X           200 non-null   int64
2   TV          200 non-null   float64
3   Radio       200 non-null   float64
4   Newspaper   200 non-null   float64
5   Web         200 non-null   float64
6   Sales       200 non-null   float64
dtypes: float64(5), int64(2)
memory usage: 11.1 KB
```

In [15]: *#Separar variable respuesta y predictoras*
respuesta = datos["Sales"].copy()
predictoras = datos.drop(["Unnamed: 0", "X", "Sales"], axis=1)

In [23]: *#Estandarizar las variables*
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(predictoras)

predictoras_est = pd.DataFrame(scaler.transform(predictoras), columns = predicto

In [25]: predictoras_est

Out[25]:

	TV	Radio	Newspaper	Web
0	0.969852	0.981522	1.778945	1.919103
1	-1.197376	1.082808	0.669579	1.867138
2	-1.516155	1.528463	1.783549	-1.436755
3	0.052050	1.217855	1.286405	1.281985
4	0.394182	-0.841614	1.281802	0.470782
...	...	...	...	...
195	-1.270941	-1.321031	-0.771217	1.164843
196	-0.617035	-1.240003	-1.033598	-0.542207
197	0.349810	-0.942899	-1.111852	0.700669
198	1.594565	1.265121	1.640850	1.016806
199	0.993206	-0.990165	-1.005979	-0.099143

200 rows × 4 columns

In [29]:

```
#Importar clase
from sklearn.linear_model import Ridge

#Ajustar el modelo ridge con alfa=10
ridge_reg = Ridge(alpha = 10)
ridge_reg.fit(predictoras_est, respuesta)

#Obtener coeficientes del modelo
#Intercepto
print("El intercepto es %s " %ridge_reg.intercept_)

#Coeficientes de regresion
coefs_ridge_a = pd.DataFrame({"Variables": predictoras_est.columns,
                              "Coeficientes": ridge_reg.coef_})

coefs_ridge_a
```

El intercepto es 14.0225

Out[29]:

	Variables	Coeficientes
0	TV	3.731627
1	Radio	2.680016
2	Newspaper	0.041911
3	Web	0.281170

In [31]:

```
#Ajustar el modelo ridge con alfa=100

ridge_reg_b = Ridge(alpha = 100)
ridge_reg_b.fit(predictoras_est, respuesta)

#Obtener coeficientes del modelo
```

```
#Intercepto
print("El intercepto es %s " %ridge_reg_b.intercept_)

#Coeficientes de regresion
coefs_ride_b = pd.DataFrame({"Variables": predictoras_est.columns,
                             "Coeficientes": ridge_reg_b.coef_})

coefs_ride_b
```

El intercepto es 14.0225

Out[31]:

	Variables	Coeficientes
0	TV	2.635205
1	Radio	1.853438
2	Newspaper	0.260701
3	Web	0.146836

In [33]: #Ajustar el modelo ridge con alfa=1000

```
ridge_reg_c = Ridge(alpha = 1000)
ridge_reg_c.fit(predictoras_est,respuesta)

#Obtener coeficientes del modelo
#Intercepto
print("El intercepto es %s " %ridge_reg_c.intercept_)

#Coeficientes de regresion
coefs_ride_c = pd.DataFrame({"Variables": predictoras_est.columns,
                             "Coeficientes": ridge_reg_c.coef_})

coefs_ride_c
```

El intercepto es 14.0225

Out[33]:

	Variables	Coeficientes
0	TV	0.672511
1	Radio	0.484283
2	Newspaper	0.163210
3	Web	0.011891

In [35]: #Importar clase para la regresión lineal
 from sklearn.linear_model import LinearRegression

```
#Ajustar el modelo
lm = LinearRegression()
lm.fit(predictoras_est, respuesta)

#Obtener error para cada uno
y_pred_lm = lm.predict(predictoras_est)
y_pred_ride = ridge_reg.predict(predictoras_est)
y_pred_ride_b = ridge_reg_b.predict(predictoras_est)
y_pred_ride_c = ridge_reg_c.predict(predictoras_est)

mse_lm = mean_squared_error(respuesta, y_pred_lm)
mse_ride = mean_squared_error(respuesta, y_pred_ride)
```

```

mse_ridge_b = mean_squared_error(respuesta, y_pred_ridge_b)
mse_ridge_c = mean_squared_error(respuesta, y_pred_ridge_c)
errores_ridge = [mse_lm, mse_ridge, mse_ridge_b, mse_ridge_c]

comparación_ridge = pd.DataFrame({"Modelo": ["Lineal", r"Ridge $\alpha=10$", r"Ridge $\alpha=100$", r"Ridge $\alpha=1000$"],
                                  "MSE": errores_ridge})

comparación_ridge

```

Out[35]:

	Modelo	MSE
0	Lineal	2.690922
1	Ridge $\alpha=10$	2.744897
2	Ridge $\alpha=100$	5.253657
3	Ridge $\alpha=1000$	19.133682

El modelo de regresión lineal presenta el menor MSE con 2.6909, lo que indica que es el modelo con mejor capacidad predictiva entre los evaluados. El modelo Ridge con $\alpha=10$ y MSE = 2.7449 muestra un desempeño muy cercano al modelo lineal, lo que sugiere que una regularización leve no afecta significativamente la calidad del ajuste. Sin embargo, los modelos Ridge con MSE = 5.2537 y MSE = 19.1337 presentan un incremento considerable en el error, lo que indica que valores más altos del parámetro de regularización provocan un deterioro en la capacidad predictiva.

In [37]:

```

fig = plt.figure(figsize=(6, 6))

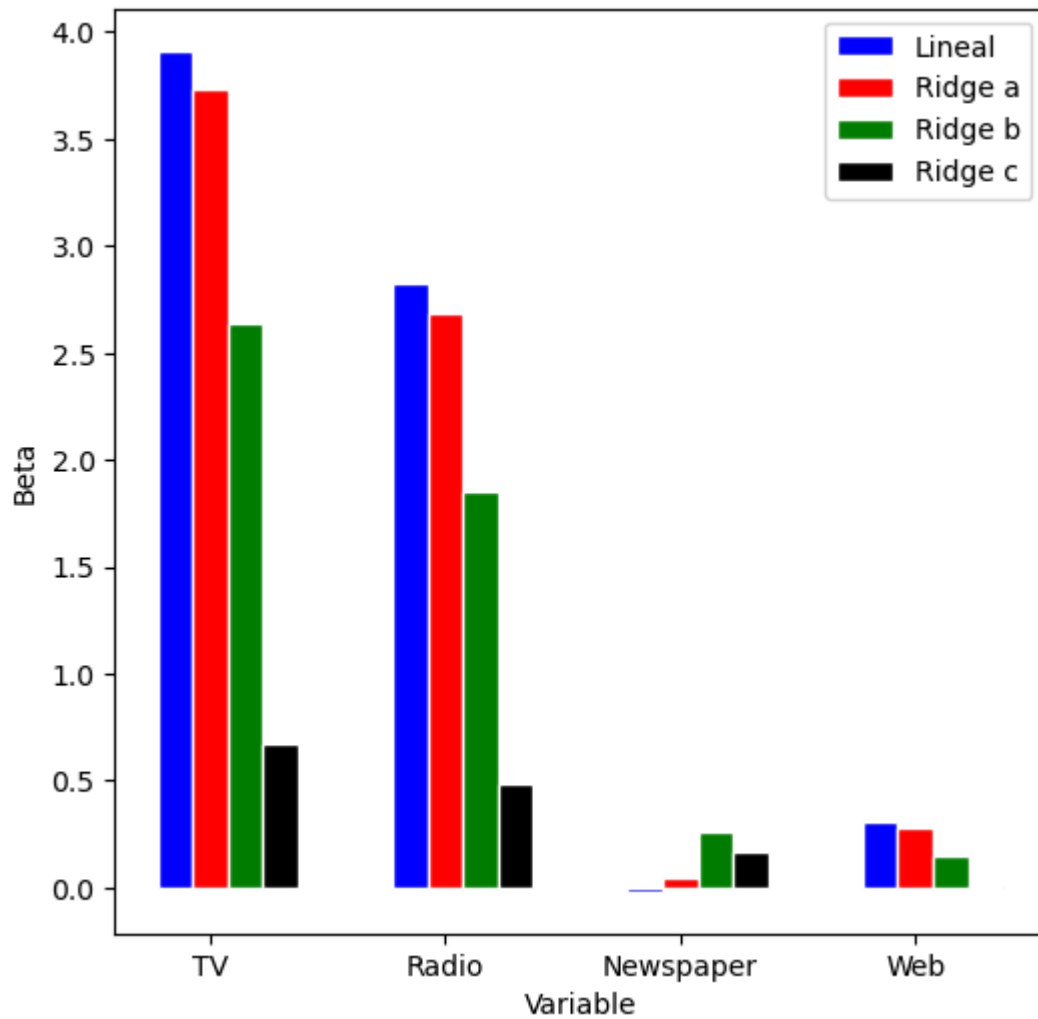
#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
r4 = [x + barWidth for x in r3]

#Pintar las barras
plt.bar(r1, lm.coef_, color = "blue", width = barWidth, edgecolor = "white", label="Lineal")
plt.bar(r2, ridge_reg.coef_, color = "red", width = barWidth, edgecolor = "white", label="Ridge $\alpha=10$")
plt.bar(r3, ridge_reg_b.coef_, color = "green", width = barWidth, edgecolor = "white", label="Ridge $\alpha=100$")
plt.bar(r4, ridge_reg_c.coef_, color = "black", width = barWidth, edgecolor = "white", label="Ridge $\alpha=1000$")

plt.xticks([r + barWidth for r in range(len(lm.coef_))],
            [predictoras.columns[i] for i in range(len(predictoras.columns))]);
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)

```



```
In [39]: from sklearn.linear_model import RidgeCV
from sklearn.model_selection import KFold, cross_val_score

alphas = np.logspace(-3, 4, 100)
ridge = RidgeCV(alphas=alphas, cv=10)
ridge.fit(predictoras_est, respuesta)
kf = KFold(n_splits=10, shuffle=True, random_state=2026)

mse_mean = list()
mse_std = list()

for alpha in alphas:
    modelo_tmp = RidgeCV(alphas=[alpha], cv=None)
    scores = cross_val_score(modelo_tmp, predictoras_est, respuesta, scoring='neg_log_loss')

    mse = -scores
    mse_mean.append(mse.mean())
    mse_std.append(mse.std())

mse_mean = np.array(mse_mean)
mse_std = np.array(mse_std)

alpha_min = ridge.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]
```

```

idx_1se = np.where(mse_mean <= mse_min + se_min)[0][0]
alpha_1se = alphas[idx_1se]

#Realizar el gráfico
x = np.log(alphas)

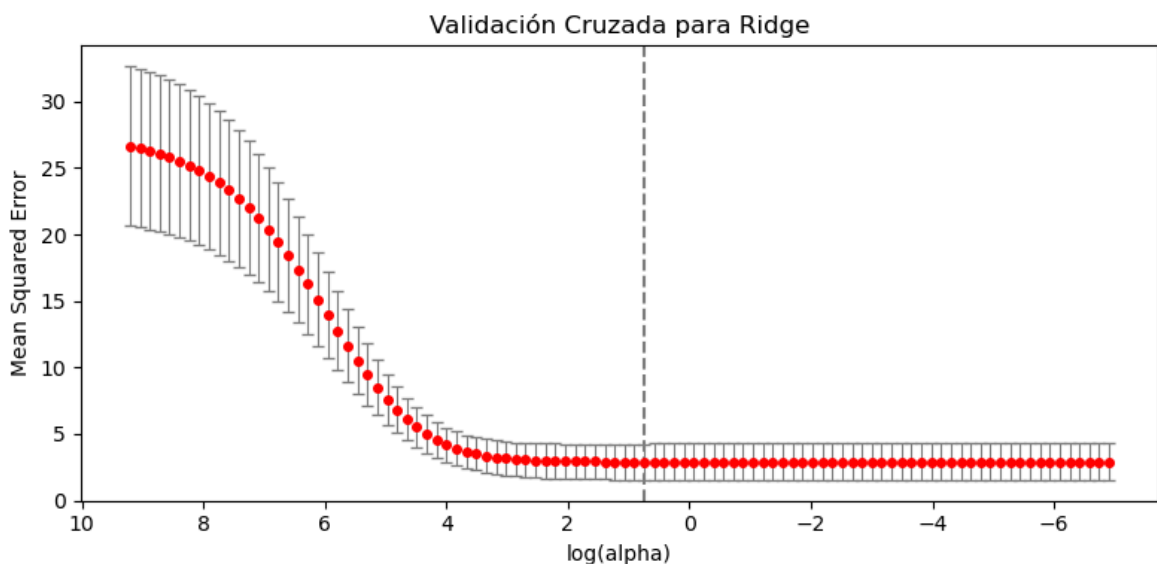
plt.figure(figsize=(8, 4))
plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', color='red', ecolor='gray', eli
plt.axvline(np.log(alpha_min), linestyle='--', color="gray")

plt.xlabel("log(alpha)")
plt.ylabel('Mean Squared Error')
plt.title('Validación Cruzada para Ridge')
plt.gca().invert_xaxis()

plt.tight_layout()
plt.show()

print("El valor de alpha que minimiza el mse es %s" %alpha_min )

```



El valor de alpha que minimiza el mse es 2.1049041445120196

```

In [41]: #Importar clase
from sklearn.linear_model import Ridge

#Ajustar el modelo
ridge_reg_d = Ridge(alpha = alpha_min, solver = "auto")
ridge_reg_d.fit(predictoras_est, respuesta)

#Obtener coeficientes del modelo
# intercepto
print(ridge_reg_d.intercept_)
# coeficientes de regresion
print(ridge_reg_d.coef_)

```

14.0225

[3.87343596e+00 2.79550139e+00 -3.69024163e-03 3.01801693e-01]

```

In [43]: #Obtener error para cada uno
y_pred_ridge_d = ridge_reg_d.predict(predictoras_est)

mse_ridge_d = mean_squared_error(respuesta, y_pred_ridge_d)

```

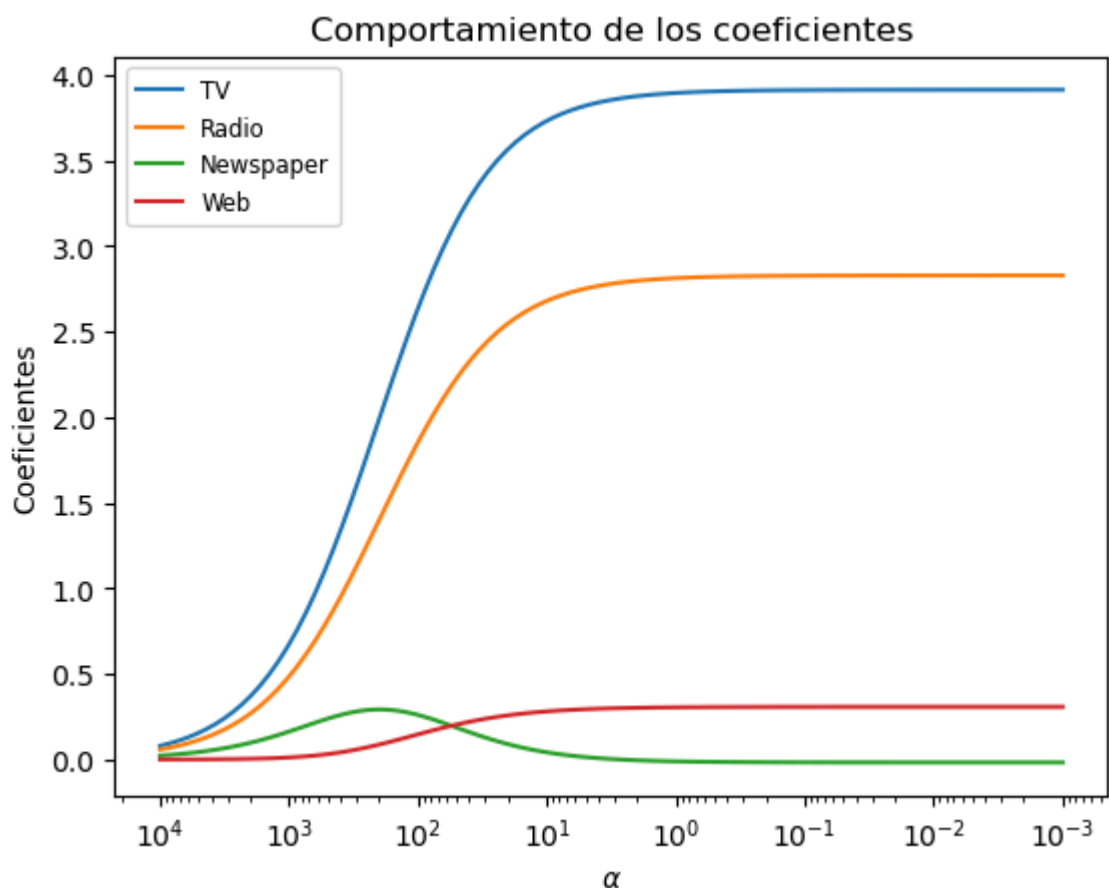
In [45]: `mse_ridge_d`

Out[45]: 2.6935219526274072

```
In [47]: coefs = list()
for a in alphas:
    ridge = linear_model.Ridge(alpha=a, fit_intercept=False)
    ridge.fit(predictoras_est, respuesta)
    coefs.append(ridge.coef_)
```

```
In [49]: ax = plt.gca()

ax.plot(alphas, coefs)
ax.set_xscale("log")
ax.set_xlim(ax.get_xlim()[::-1]) # reverse axis
plt.xlabel(r"$\alpha$")
plt.ylabel("Coeficientes")
plt.title("Comportamiento de los coeficientes")
plt.axis("tight")
plt.legend([f"Feature {i + 1}" for i in range(predictoras_est.shape[1])], loc="b")
plt.legend([predictoras.columns[i] for i in range(predictoras_est.shape[1])], loc="b")
plt.show()
```



Ejemplo 1 - Parte 2

Realizar un modelo de regresión Lasso del volúmenes de las ventas con las variables restantes.

a) Con el parámetro de encogimiento $\alpha=0.01$

- b) Con el parámetro de encogimiento $\alpha=0.5$
- c) Con el parámetro de encogimiento $\alpha=1$
- d) Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?
- e) Realizar la validación cruzada para determinar el valor de α que proporciona un menor error cuadrático
- f) Comparar el comportamiento de los coeficientes cuando α disminuye y cuando α tiende a hacerse más grande.

```
In [56]: #Importar clase
from sklearn.linear_model import Lasso

#Ajustar el modelo de Lasso con alfa=0.05
lasso_reg = Lasso(alpha=0.05)
lasso_reg.fit(predictoras_est, respuesta)
#Intercepto
print("El intercepto es %s" %lasso_reg.intercept_)
#Coeficientes de regresion

pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": lasso_reg.coef_})
```

El intercepto es 14.0225

Out[56]:

	Variables	Coeficientes
0	TV	3.866135
1	Radio	2.767894
2	Newspaper	0.000000
3	Web	0.251896

0	TV	3.866135
1	Radio	2.767894
2	Newspaper	0.000000
3	Web	0.251896

```
In [58]: #Ajustar el modelo de Lasso con alfa=0.1

lasso_reg_b = Lasso(alpha=0.1)
lasso_reg_b.fit(predictoras_est, respuesta)
#Intercepto
print("El intercepto es %s" %lasso_reg_b.intercept_)
#Coeficientes de regresion

pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": lasso_reg_b.coef_})
```

El intercepto es 14.0225

Out[58]:

	Variables	Coeficientes
0	TV	3.819818
1	Radio	2.713552
2	Newspaper	0.000000
3	Web	0.195812

0	TV	3.819818
1	Radio	2.713552
2	Newspaper	0.000000
3	Web	0.195812

```
In [60]: #Ajustar el modelo de Lasso con alfa=1
lasso_reg_c = Lasso(alpha = 1)
lasso_reg_c.fit(predictoras_est, respuesta)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %lasso_reg_c.intercept_)
#Coeficientes de regresion
coefs_lasso_c = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
coefs_lasso_c
```

El intercepto que se tiene es 14.0225

Out[60]:

	Variables	Coeficientes
0	TV	2.970373
1	Radio	1.836089
2	Newspaper	0.000000
3	Web	0.000000

```
In [62]: #Obtener error para cada uno
y_pred_lasso = lasso_reg.predict(predictoras_est)
y_pred_lasso_b = lasso_reg_b.predict(predictoras_est)
y_pred_lasso_c = lasso_reg_c.predict(predictoras_est)

mse_lasso = mean_squared_error(respuesta, y_pred_lasso)
mse_lasso_b = mean_squared_error(respuesta, y_pred_lasso_b)
mse_lasso_c = mean_squared_error(respuesta, y_pred_lasso_c)
errores_lasso = [mse_lm, mse_lasso, mse_lasso_b, mse_lasso_c]

comparación_lasso = pd.DataFrame({"Modelo": ["Lineal", r"Lasso $\alpha=0.05$", r
"MSE": errores_lasso})

comparación_lasso
```

Out[62]:

	Modelo	MSE
0	Lineal	2.690922
1	Lasso $\alpha=0.05$	2.744897
2	Lasso $\alpha=0.1$	5.253657
3	Lasso $\alpha=1$	19.133682

El modelo de regresión lineal presenta el menor MSE con 2.6909, lo que indica que es el modelo con mejor capacidad predictiva. El modelo Lasso con $\alpha=0.05$ MSE = 2.7449 muestra un desempeño muy cercano al modelo lineal, lo que sugiere que una penalización leve mantiene la precisión del modelo mientras empieza a simplificarlo. Sin embargo, los modelos Lasso con MSE = 5.2537 y MSE = 19.1337 presentan un incremento considerable en el error, lo que indica que valores más altos del parámetro de regularización

```
In [64]: from sklearn.linear_model import LassoCV, lasso_path

# Ajustar LassoCV
lasso_cv = LassoCV(cv=10, random_state=2026, n_alphas=100)
lasso_cv.fit(predictoras_est, respuesta)

alphas = lasso_cv.alphas_
mse_path = lasso_cv.mse_path_
mse_mean = mse_path.mean(axis=1)
mse_std = mse_path.std(axis=1)
alpha_min = lasso_cv.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]
idx_1se = np.where(mse_mean <= mse_min + se_min)[0][0]
alpha_1se = alphas[idx_1se]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(predictoras_est)
alphas_path, coefs_path, _ = lasso_path(X_scaled, respuesta, alphas=alphas)
n_nonzero = np.sum(coefs_path != 0, axis=0)

x = np.log(alphas)

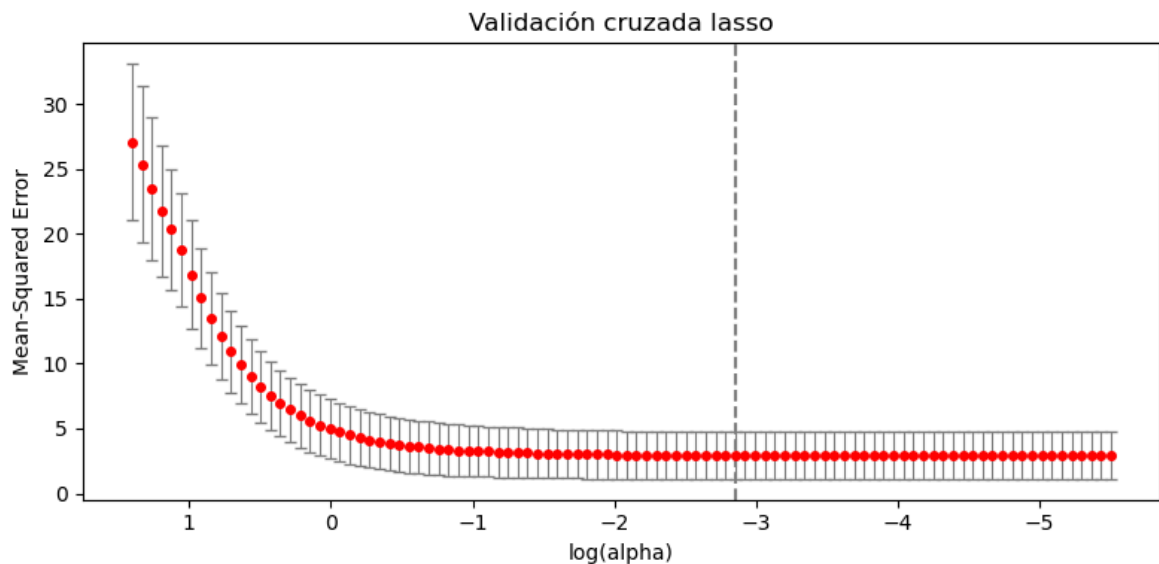
fig, ax = plt.subplots(figsize=(8, 4))

plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', color='red', ecolord='gray', eli

ax.set_xlabel('log(alpha)')
ax.set_ylabel('Mean-Squared Error')
ax.set_title('Validación cruzada lasso')
ax.invert_xaxis()

plt.axvline(np.log(alpha_min), linestyle='--', color="gray")
plt.tight_layout()
plt.show()

print("El valor de alpha que minimiza el mse es %s" %alpha_min )
```



El valor de alpha que minimiza el mse es 0.057705459931336404

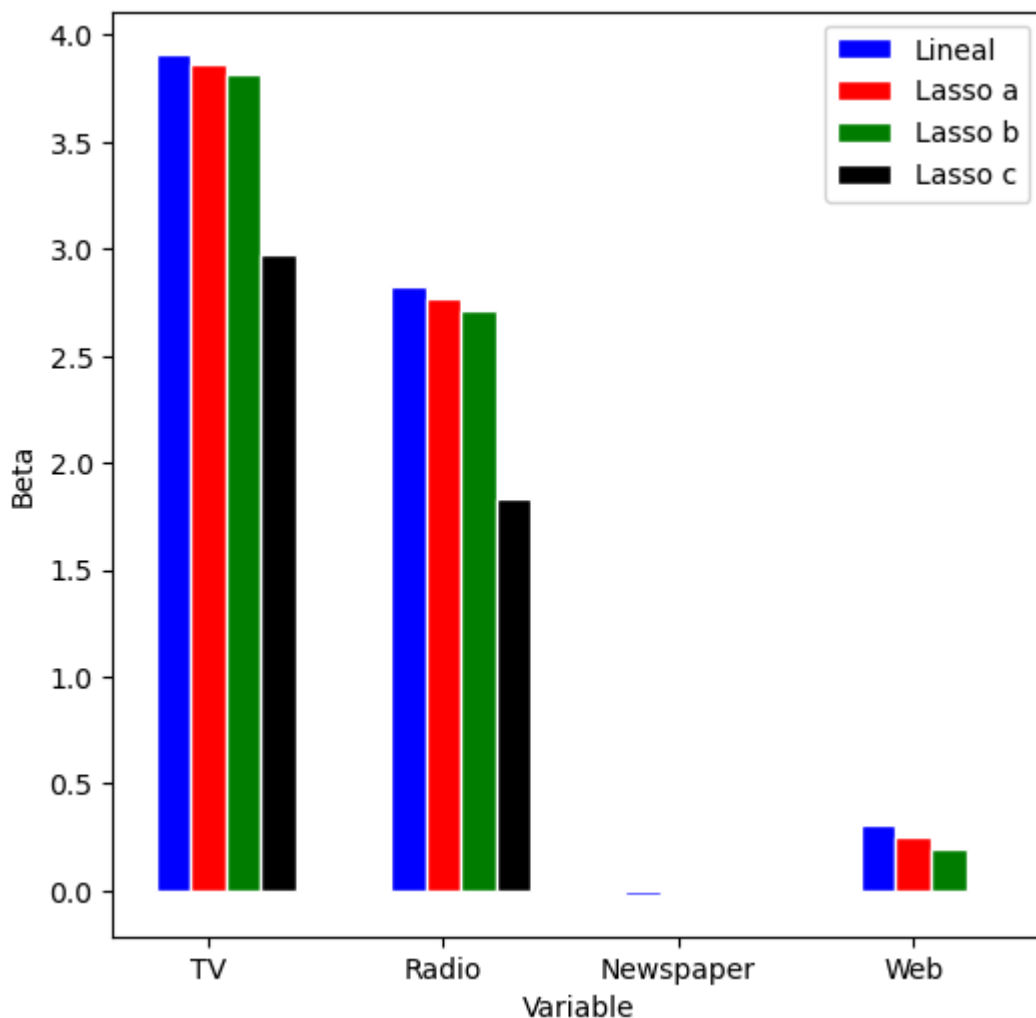
```
In [66]: fig = plt.figure(figsize=(6, 6))

#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
r4 = [x + barWidth for x in r3]

#Pintar las barras
plt.bar(r1, lm.coef_, color = "blue", width = barWidth, edgecolor = "white", label="Lineal")
plt.bar(r2, lasso_reg.coef_, color = "red", width = barWidth, edgecolor = "white", label="Lasso a")
plt.bar(r3, lasso_reg_b.coef_, color = "green", width = barWidth, edgecolor = "white", label="Lasso b")
plt.bar(r4, lasso_reg_c.coef_, color = "black", width = barWidth, edgecolor = "white", label="Lasso c")

plt.xticks([r + barWidth for r in range(len(lm.coef_))],
            [predictoras.columns[i] for i in range(len(predictoras.columns))]);
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)
```



```
In [68]: coefs = list()
for a in alphas:
```

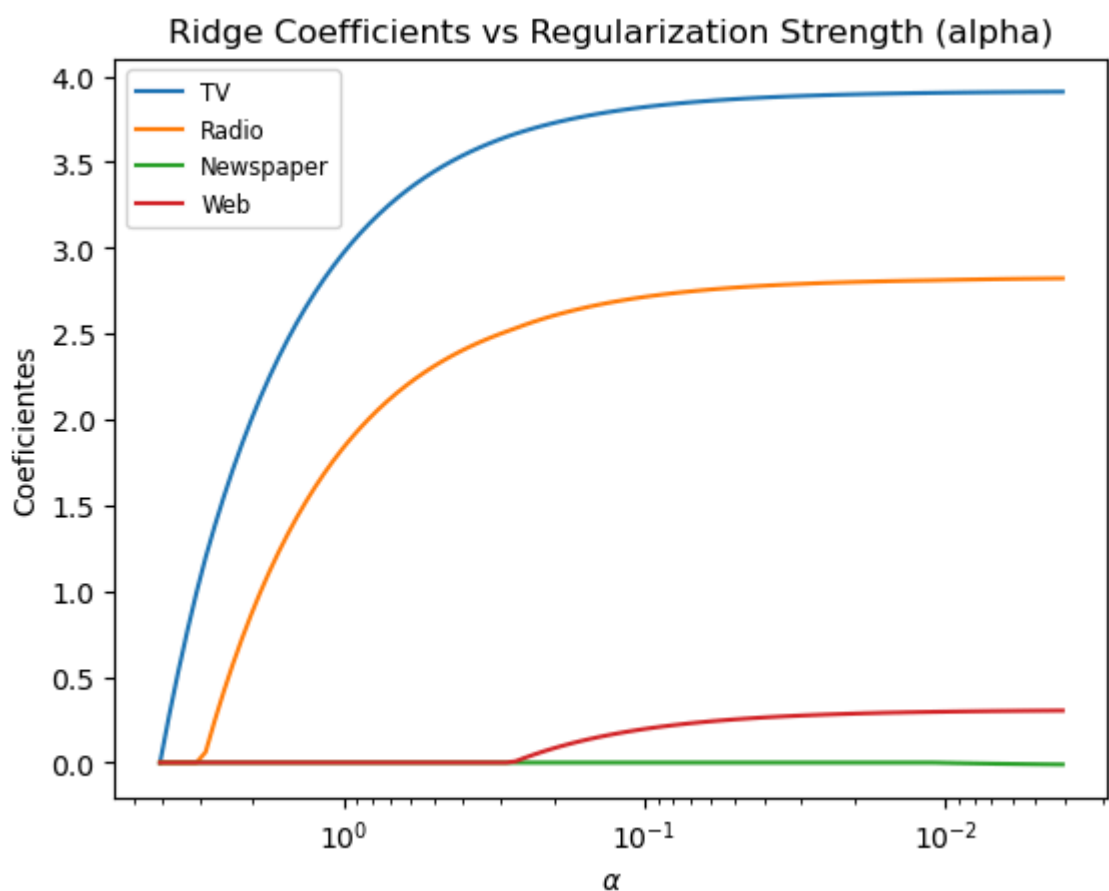
```

lasso = linear_model.Lasso(alpha=a, fit_intercept=False)
lasso.fit(predictoras_est, respuesta)
coefs.append(lasso.coef_)

ax = plt.gca()

ax.plot(alphas, coefs)
ax.set_xscale("log")
ax.set_xlim(ax.get_xlim()[::-1]) # reverse axis
plt.xlabel(r"$\alpha$")
plt.ylabel("Coeficientes")
plt.title("Ridge Coefficients vs Regularization Strength (alpha)")
plt.axis("tight")
plt.legend([f"Feature {i + 1}" for i in range(predictoras_est.shape[1])], loc="b")
plt.legend([predictoras.columns[i] for i in range(predictoras_est.shape[1])], loc="b")
plt.show()

```



Ejemplo 1 - Parte 3

Realizar un modelo de Red elástica del volúmenes de las ventas con las variables restantes.

- Fijando $r=0.1$
- Fijando $r=0.90$ (En ambos casos ajustar por validación cruzada con $k=10$ particiones)
- Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?

d) Realizar la validación cruzada para determinar el valor de α y λ que proporciona un menor error cuadrático.

```
In [70]: #Importar clase
from sklearn.linear_model import ElasticNetCV

#Ajustar el modelo con r=0.1
e_net = ElasticNetCV(cv=10, l1_ratio=0.1, random_state=2026)
e_net.fit(predictoras_est, respuesta)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %e_net.intercept_)
#Coeficientes de regresion
coefs_e_net = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": e_net.coef_})
```

El intercepto que se tiene es 14.0225

```
Out[70]:
```

	Variables	Coeficientes
0	TV	3.774890
1	Radio	2.714574
2	Newspaper	0.024483
3	Web	0.283460

	Variables	Coeficientes
0	TV	3.774890
1	Radio	2.714574
2	Newspaper	0.024483
3	Web	0.283460

```
In [72]: #Ajustar el modelo con r=0.9
e_net_b = ElasticNetCV(cv = 10, l1_ratio = 0.9, random_state=2026)
e_net_b.fit(predictoras_est, respuesta)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %e_net_b.intercept_)
#Coeficientes de regresion
coefs_e_net_b = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": e_net_b.coef_})
```

El intercepto que se tiene es 14.0225

```
Out[72]:
```

	Variables	Coeficientes
0	TV	3.854052
1	Radio	2.762064
2	Newspaper	0.000000
3	Web	0.256437

	Variables	Coeficientes
0	TV	3.854052
1	Radio	2.762064
2	Newspaper	0.000000
3	Web	0.256437

```
In [74]: #Obtener error para cada uno
y_pred_e_net = e_net.predict(predictoras_est)
y_pred_e_net_b = e_net_b.predict(predictoras_est)

mse_e_net = mean_squared_error(respuesta, y_pred_e_net)
mse_e_net_b = mean_squared_error(respuesta, y_pred_e_net_b)
```

```
errores_e_net = [mse_lm, mse_e_net, mse_e_net_b]

comparación_e_net = pd.DataFrame({"Modelo": ["Lineal", r"Red elástica $r=0.1$", r"Red elástica $r=0.9$"],
                                "MSE": errores_e_net})

comparación_e_net
```

Out[74]:

	Modelo	MSE
0	Lineal	2.690922
1	Red elástica \$r=0.1\$	2.744897
2	Red elástica \$r=0.9\$	5.253657

El modelo de regresión lineal presenta el menor MSE (2.6909), lo que indica que mantiene la mejor capacidad predictiva. El modelo de Red Elástica con $r=0.1$ tiene MSE = 2.7449, muestra un desempeño muy cercano al modelo lineal, lo que sugiere que una combinación adecuada de regularización de Ridge y Lasso permite mantener la precisión mientras se controla la complejidad del modelo.

Sin embargo, el modelo de Red Elástica con $r=0.9$ con MSE = 5.2537 presenta un aumento considerable en el error, indicando que una penalización más fuerte provoca una pérdida de capacidad predictiva.

In [76]:

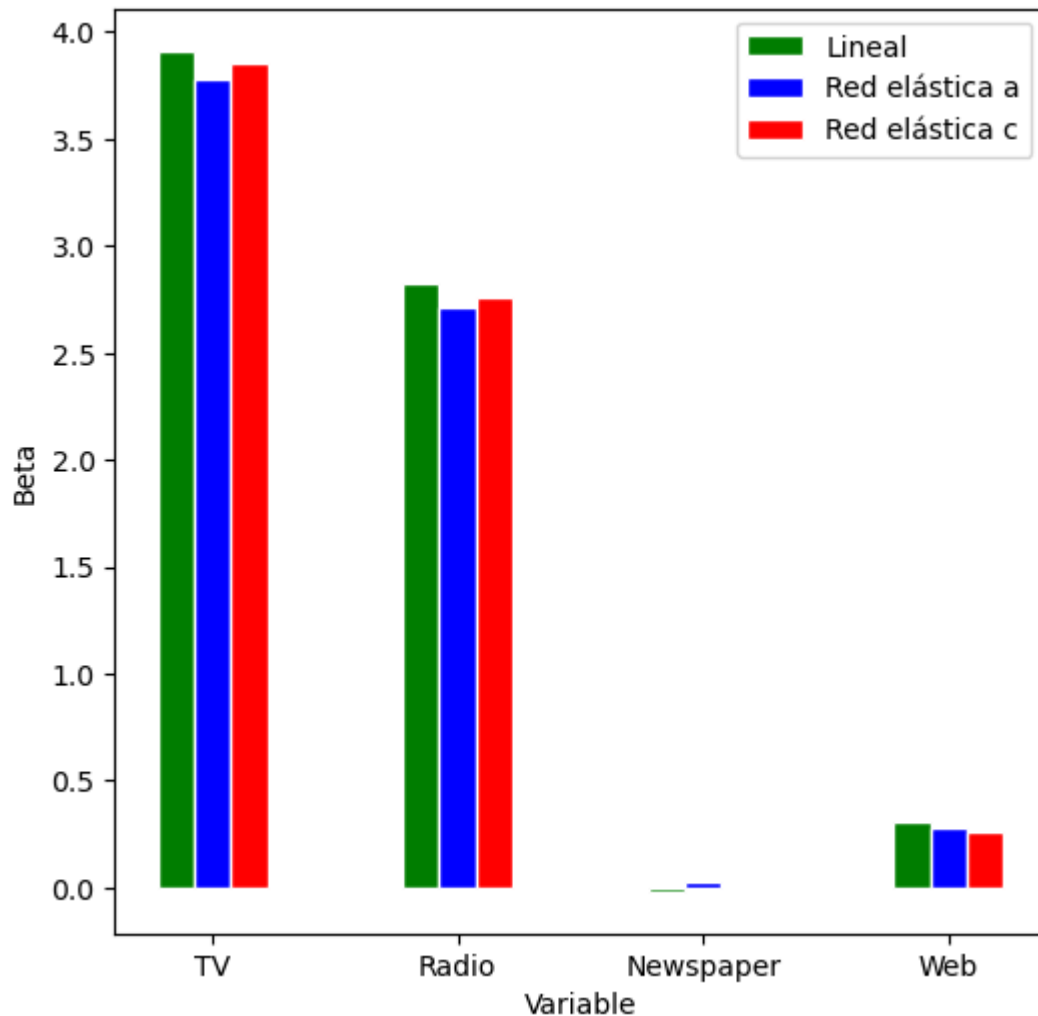
```
fig = plt.figure(figsize=(6, 6))

#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]

#Pintar Las barras
plt.bar(r1, lm.coef_, color = "green", width = barWidth, edgecolor = "white", label="Lineal")
plt.bar(r2, e_net.coef_, color = "blue", width = barWidth, edgecolor = "white", label="Red elástica $r=0.1$")
plt.bar(r3, e_net_b.coef_, color = "red", width = barWidth, edgecolor = "white", label="Red elástica $r=0.9$")

plt.xticks([r + barWidth for r in range(len(lm.coef_))],
            [predictoras.columns[i] for i in range(len(predictoras.columns))]);
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)
```



```
In [78]: modelo_cv = ElasticNetCV(alphas=alphas,l1_ratio=np.linspace(0.1, 0.99, 10),
cv=10, max_iter=1000, random_state=2026)

modelo_cv.fit(predictoras_est, respuesta)

print("El valor de alfa que minimiza el error es %s" %modelo_cv.alpha_)
print("El valor de r que minimiza el error es %s" %modelo_cv.l1_ratio_)
```

El valor de alfa que minimiza el error es 0.057705459931336404

El valor de r que minimiza el error es 0.99

```
In [80]: import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import ElasticNet, ElasticNetCV
from sklearn.model_selection import KFold, cross_val_score

# Parámetros
r = 0.99

# Modelo CV
enet_cv = ElasticNetCV( alphas=alphas,l1_ratio=r,cv=10, max_iter=1000, random_st
enet_cv.fit(predictoras_est, respuesta)
kf = KFold(n_splits=10, shuffle=True, random_state=2026)

mse_mean = list()
mse_std = list()
```



```

for alpha in alphas:
    modelo = ElasticNet(alpha=alpha, l1_ratio=r, max_iter=1000)
    scores = cross_val_score(modelo, predictoras_est, respuesta, scoring='neg_mean_
    mse = -scores
    mse_mean.append(mse.mean())
    mse_std.append(mse.std())

mse_mean = np.array(mse_mean)
mse_std = np.array(mse_std)

alpha_min = enet_cv.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]

idx_1se = np.where(mse_mean <= mse_min + se_min)[0][-1]
alpha_1se = alphas[idx_1se]

x = np.log(alphas)

#Realizar el gráfico
plt.figure(figsize=(8, 4))

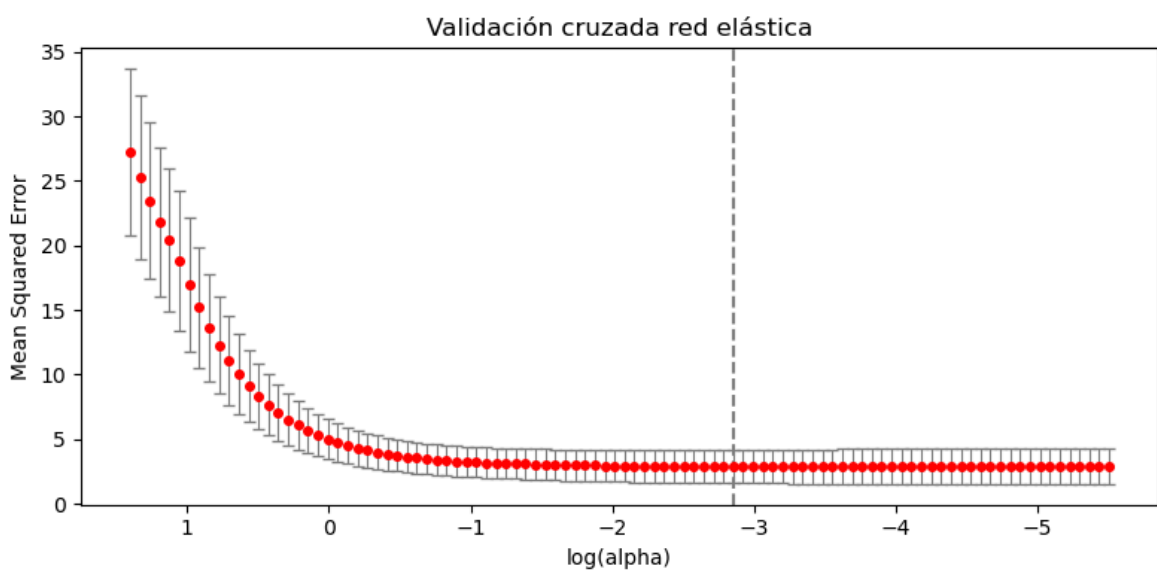
plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', ecolor='gray', elinewidth=0.8,
plt.axvline(np.log(alpha_min), linestyle='--', color='gray')

plt.xlabel("log(alpha)")
plt.ylabel("Mean Squared Error")
plt.title("Validación cruzada red elástica")
plt.gca().invert_xaxis()

plt.tight_layout()
plt.show()

print("alfa_min = ", alpha_min)

```



alfa_min = 0.057705459931336404

In []: